

Repeat-Order Memory + Auto-Quotation Engine

Systems 1 & 2 from the AI Systems Blueprint · Patel Heaters & Control Pvt. Ltd.

Problem attacked	Repeat-order spec hunting (2–3 day dispatch delays) + quotation pricing calculated by hand
Build duration	1 week
Phase	Phase 1 — the daily bleeding
Prepared by	BuildOur AI

1. The Problem, Precisely

When a regular customer asks for a repeat order — and roughly 70% of customers are regular, most re-ordering the same items — the team digs through old specifications and past drawings by hand. This happens almost every week, delays dispatch by 2–3 days, and sometimes costs the follow-up order from that customer. On top of that, every quotation's price is calculated manually, whether the item was quoted last month or last year.

Mitesh's own success definition: *"repeat orders confirmed within minutes instead of waiting for my team to check old files."* This product is that sentence, built.

2. How It Will Work

One flow, five steps, from message to quotation:

- 1

Inquiry lands. A customer message arrives on the business WhatsApp number or inquiry email (95% of inquiries already come through these two channels).

2 The system recognises the customer. It matches the sender against the customer master and pulls their complete history: every past item, spec, drawing reference, agreed price, and delivery time.

3 It matches the request. "Same 2kW cartridge heater as last time" resolves to the exact item, spec, and drawing from the customer's own history — no one opens an old file, register, or email thread.

4 It prices and drafts. For a recognised repeat item, the quotation generates instantly in the standard format with terms and conditions, using the last agreed price. For items needing calculation, the encoded costing logic runs and produces a priced draft.

5 It sends — or asks. Per Mitesh's guardrails:

- **Repeat order, regular customer, unchanged spec** → the system may confirm and send on its own.
- **New customer, new item, changed spec, or any uncertainty** → marked "**Needs Review**", routed to the concerned product manager, and nothing reaches the customer until a person approves. The AI never guesses a heater specification.

Every inquiry, match, and quote is logged against the customer automatically — so the system gets more complete with every order it touches.

3. Prerequisites (before build week starts)

These are the inputs the build consumes. Without them the week stalls — this is the Track 0 dependency from the main blueprint, scoped down to only what Systems 1 & 2 need:

1. **Top repeat customers digitised first.** The customer list ranked by order frequency; for the top slice: their standard items, last agreed prices, specifications, and drawings collected into one place (drawings are currently at 0% in Odoo, customer database at 20% — this is the gap being closed).

2. **The costing logic captured.** However pricing is calculated today — formula, thumb rules, costing sheet, or what sits in a manager's head — written down once so the engine can encode it.
3. **The standard quotation format** and terms & conditions, as a template.
4. **One official inquiry channel per medium.** A business WhatsApp number and a designated inquiry mailbox — inquiries must stop landing only on personal numbers.
5. **The auto-confirm list.** Mitesh names which regular customers the system may confirm on its own, and which stay human-approved regardless.
6. **Odoo access** for reading and writing customer, product, and quotation records.

Who does the data work: Tannu's CRM team (8 people) — this is the delegation the main blueprint calls for, since Mitesh has 6 hrs/week.

4. Implementation — One Week

DAY	WHAT HAPPENS
Day 1	Load the digitised customer history (top repeat customers) into the system; connect the inquiry channels.
Day 2	Build customer recognition + history retrieval: incoming message → customer identified → full history on one screen.
Day 3	Encode the costing logic and quotation template; generate first draft quotations from real past inquiries and compare against what was actually sent.
Day 4	Wire the guardrail routing: auto-confirm rules, "Needs Review" flags, product-manager approval queue.
Day 5	Dry run on live inquiries in shadow mode — the system drafts, the team still sends manually, every mismatch is corrected and fed back.
Day 6	Go live for the auto-confirm list; human-approval flow live for everyone else. Team walkthrough with Tannu's team and the product managers.
Day 7	Review day with Mitesh: what auto-confirmed, what got flagged, what needs tuning. Guardrail adjustments locked in.

(A detailed technical build plan — stack, integrations, hosting — will be a separate document, as agreed.)

5. The Outcome

When the week ends:

- A regular customer's repeat order is **confirmed in minutes**, not after 2–3 days of file hunting.
- Quotations go out in the **standard format with T&C, priced by the engine** — manual price calculation is off the "never again" list for every encoded item.
- Nothing wrong reaches a customer: anything uncertain is flagged **"Needs Review"** and waits for a person.
- Every inquiry that flows through **enriches the customer database automatically** — the 20% coverage problem starts fixing itself as a side effect of daily work.

What it does **not** do (by design): price new customers without human sign-off, message key accounts without clearance, or share drawings, custom designs, or special pricing externally.

6. The Bigger Picture

This build is deliberately first because it pays for everything after it:

- **It attacks the leak Mitesh ranked #1** — the daily, "uncountable"-cost delay on the exact orders that are the business's bread and butter. The 70% repeat base feels the improvement immediately, in the season run-up when it matters most.
- **It frees the people who are double-hatting.** Supervisors currently break off production work to chase old specs and make follow-up calls. Every hour returned goes back into production and customer visits.
- **It starts removing Mitesh as the routing layer.** Repeat business — the bulk of daily decisions — stops needing him. His 6 hrs/week go to approvals and guardrails instead of reminders and file searches.
- **It builds the data foundation the rest of the blueprint stands on.** The customer/spec/drawing library assembled for this product is the same data the One-Window Status Agent (System 3), the QC Report Generator (System 4), and the dealer/supplier portals

(System 7) will read. Track 0 stops being an abstract "digitisation project" and becomes a working product from week one.

- **It proves the pattern for a hesitant team.** Mitesh flagged that staff "stick at one place" with new tools. The first thing this system does is remove a chore everyone hates — the strongest possible adoption argument for every phase that follows.

7. Roadmap

STEP	WINDOW	WHAT HAPPENS	EXIT CONDITION
Prerequisites	Before build week	Tannu's team digitises top repeat customers; costing logic and quote template captured; channels and auto-confirm list agreed	Inputs from §3 ready
Build week	Days 1–7	The implementation plan above	Repeat orders auto-confirming for the approved list; drafts + approval queue for the rest
Parallel run	Week 2	System handles live repeat inquiries; team spot-checks every auto-confirmation; mismatches tuned	A full week with no wrong spec reaching a customer
Widen coverage	Weeks 3–4	Next tier of repeat customers digitised and switched on; more items encoded into the costing engine	Majority of repeat volume flowing through the system
Hand off to Phase 2	From week 4	The order data this system now captures becomes the input for the Status Agent (System 3) and the task/accountability layer (System 6)	Phase 2 build begins on live data

The through-line: **before the pre-winter peak**, the entire repeat base quotes and confirms without anyone opening an old file — and the data collected on the way has already laid the rails for the rest of the blueprint.

Grounded in the Systems Brief (01.08.2026) and the AI Systems Blueprint. Duration and sequencing as agreed with BuildOur AI; no client-side figures added beyond those Mitesh provided.